

BOAS PRÁTICAS NA ORGANIZAÇÃO DE APIS

Sumário:

1. Arquitetura em Camadas e Separação de Responsabilidades.....	2
1.1. Arquitetura em Camadas — O que é e como funciona?.....	2
1.2. Separação de Responsabilidades (SoC) — Por que é tão importante?.....	3
1.3. Como esses dois conceitos funcionam juntos?.....	4
2. Sugestão de Organização para APIs.....	5
2.1 Criação e Configuração Inicial de um projeto de exemplo.....	5
2.2 Criando a Base de Dados.....	7
2.3 Configuração do aplicativo Express.....	9
2.3 Configuração do servidor.....	10
2.4 Model.....	11
2.5 Service.....	13
2.6 Controller.....	14
2.7 Rotas.....	16
2.8 Middleware de Tratamento de Erros.....	17
2.9 Utils.....	18
2.10 Database.....	19
2.11 Testando as Rotas do Servidor HTTP.....	20

1. Arquitetura em Camadas e Separação de Responsabilidades

Quando desenvolvemos APIs profissionais, não basta “fazer funcionar”. Em projetos reais, as empresas esperam sistemas organizados, fáceis de manter e que possam crescer com o tempo. Para isso, usamos duas ideias muito importantes:

- **Arquitetura em Camadas (Layered Architecture)**
- **Separação de Responsabilidades (Separation of Concerns – SoC)**

Esses conceitos ajudam a deixar o código mais limpo, organizado e profissional — e são amplamente usados no mercado.

1.1. Arquitetura em Camadas — O que é e como funciona?

A Arquitetura em Camadas é uma forma de organizar o projeto dividindo o código em partes (camadas). Cada camada tem uma função específica e conversa apenas com a camada “ao lado”. Assim, não misturamos tudo no mesmo arquivo.

Em uma API feita com Node.js e Express, normalmente temos:

1. Routes (Rotas)

Recebem as requisições (GET, POST, PUT, DELETE) e chamam o *controller* certo.

2. Controllers

São responsáveis por interpretar os dados enviados pelo usuário e acionar o *service* necessário. É como um “chefe de tráfego”: recebe algo e encaminha para o lugar certo.

3. Services

Aqui fica a **lógica de negócio**, ou seja, as regras da aplicação.
Exemplos: validar dados, calcular preços, verificar se um usuário já existe, etc.

4. Models

São os responsáveis por conversar **diretamente com o banco de dados** (MySQL).
É onde fazemos SELECT, INSERT, UPDATE etc.

Outras partes que podem existir

- **Middleware:** segurança, autenticação, logs, tratamento de erros.
- **Config:** guardar configurações como variáveis de ambiente e conexão com o banco.
- **Utils:** funções auxiliares (ex.: gerar números aleatórios, formatar datas).

Por que usar essa arquitetura?

- O projeto fica mais fácil de entender.
- Alterar algo em uma parte não quebra o sistema inteiro.
- Facilita testes e organização do código.

1.2. Separação de Responsabilidades (SoC) — Por que é tão importante?

A Separação de Responsabilidades (Separation of Concerns) é um princípio que diz que **cada parte do sistema deve ter apenas uma responsabilidade**.

Isso significa que:

- Um *controller* **não** deve acessar diretamente o banco.
- Um *model* **não** deve fazer validações.
- Um *service* **não** deve devolver respostas HTTP.

Quando cada parte do sistema tem **só uma tarefa**, temos:

- Menos erros: Porque o código não fica misturado.
- Mais organização: Sabemos exatamente onde procurar.
- Facilita manutenção: Atualizar uma regra de negócio afeta apenas o *service*, não o *controller*, rota ou *model*.
- Melhor testabilidade: Você pode testar um *service* sozinho, sem precisar subir o servidor.
- Escalabilidade: O projeto cresce sem virar uma bagunça.

1.3. Como esses dois conceitos funcionam juntos?

Podemos pensar assim:

- **A Arquitetura em Camadas organiza o projeto fisicamente** — separando rotas, controllers, services e models em pastas e arquivos.
- **A Separação de Responsabilidades garante que cada parte faça apenas o que deve fazer** — mantendo cada camada responsável por apenas um tipo de tarefa.

Exemplo prático (criação de usuário):

Camada	O que faz
Rota	Recebe a requisição POST <code>/users</code>
Controller	Pega os dados enviados e chama o service
Service	Valida e aplica regras (ex.: email já existe?)
Model	Insere o novo usuário no banco de dados

Assim, nada fica misturado. Cada parte sabe exatamente o que fazer.

2. Sugestão de Organização para APIs

Através da aplicação dos conceitos vistos nesta aula, vamos estruturar nossos projetos Node.js de forma limpa e eficiente.

2.1 Criação e Configuração Inicial de um projeto de exemplo

Primeiro, crie e acesse uma pasta para este projeto de exemplo.

```
mkdir projeto-api  
cd projeto-api
```

Em seguida, inicie o projeto criando o arquivo **package.json**, que será responsável por gerenciar as dependências e configurações do projeto.

```
npm init -y
```

Com o projeto inicializado, instale os **pacotes essenciais** que serão usados em produção e desenvolvimento:

```
npm install express mysql2 dotenv cors helmet
```

Vamos ver o papel de cada um:

- **[express](#)**: Framework web minimalista para Node.js. Permite criar rotas, servidores HTTP, middlewares e muito mais.
- **[mysql2](#)**: Driver para conectar o Node.js a um banco de dados MySQL.
- **[dotenv](#)**: Carrega variáveis de ambiente a partir de um arquivo .env (como credenciais do banco e número da porta).
- **[cors](#)**: Middleware que ativa o CORS (Cross-Origin Resource Sharing), permitindo que APIs sejam acessadas de outros domínios.
- **[helmet](#)**: Middleware de segurança que adiciona headers HTTP para proteger contra ataques comuns (XSS, clickjacking, etc.).

Em seguida, instale os **pacotes apenas para desenvolvimento** neste projeto:

```
npm install nodemon --save-dev
```

O **[nodemon](#)** é uma ferramenta que reinicia automaticamente o servidor toda vez que você altera um arquivo no projeto.

Abra o arquivo **package.json** e, na seção "scripts", adicione o script "dev".

```
...  
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
},  
...
```

Em seguida, crie a seguinte organização de pastas e arquivos neste projeto:

```
projeto-api/  
├── src/ # Diretório principal com o código-fonte da aplicação  
│   ├── config/ # Configurações gerais da aplicação  
│   │   └── database.js # Conexão com o banco de dados MySQL  
│   ├── controllers/ # Camada responsável por lidar com as requisições HTTP  
│   │   └── userController.js # Lógica de controle dos endpoints relacionados a usuários  
│   ├── database/ # Scripts SQL usados para criar e popular o banco  
│   │   ├── create_database.sql # Criação do banco de dados  
│   │   ├── create_tables.sql # Criação das tabelas necessárias (ex: users)  
│   │   └── seed_data.sql # Inserção de dados iniciais para testes  
│   ├── middlewares/ # Funções que interceptam requisições e tratam eventos específicos  
│   │   └── errorMiddleware.js # Middleware para captura e padronização de erros  
│   ├── models/ # Camada de acesso a dados (operações SQL)  
│   │   └── userModel.js # Interação com a tabela users no banco de dados  
│   ├── routes/ # Definição das rotas e associação com controllers  
│   │   └── userRoutes.js # Rotas relacionadas à entidade Usuario (GET, POST, PUT, DELETE)  
│   ├── services/ # Camada de regras de negócio (validação, verificações, etc.)  
│   │   └── userService.js # Regras para criar, atualizar, listar e remover usuários  
│   ├── tests/ # Testes manuais ou automatizados da API  
│   │   └── users_api_test.rest # Testes executáveis com a extensão REST Client do VS Code  
│   ├── utils/ # Funções auxiliares reutilizáveis  
│   │   └── validateEmail.js # Função para validar o formato de e-mails  
│   └── app.js # Configuração principal do Express (rotas, middlewares)  
└── .env # Arquivo de variáveis de ambiente (.gitignore recomendado)
```

— package.json	# Gerenciador de dependências e scripts do projeto
— package-lock.json	# Controle de versões exatas das dependências instaladas
— server.js	# Inicialização do servidor Express e carregamento do app

Observe que a estrutura do projeto segue os conceitos da Arquitetura em Camadas, pois divide claramente a aplicação em camadas funcionais (rota → controller → service → model), e respeita o princípio da Separação de Responsabilidades, garantindo que cada módulo do sistema realize apenas uma tarefa específica.

2.2 Criando a Base de Dados

Nesta aula, vamos criar um database chamado **projeto_api_db**. Para isso, execute os seguintes comandos SQL no terminal MySQL, em uma ferramenta de gerenciamento de banco de dados (como MySQL Workbench), ou em uma extensão do VS Code (como SQLTools):

```
-- Cria o banco de dados se não existir
CREATE DATABASE IF NOT EXISTS projeto_api_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

-- Usa o banco de dados
USE projeto_api_db;

-- Cria a tabela de usuários
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Inserção de dados fictícios
INSERT INTO users (name, email) VALUES
('João da Silva', 'joao@example.com'),
('Maria Oliveira', 'maria@example.com');
```

Esse script SQL realiza a configuração inicial do banco de dados para a aplicação. Primeiramente, ele cria o banco chamado **projeto_api_db** caso ainda não exista, utilizando o

conjunto de caracteres **utf8mb4**, que permite armazenar corretamente acentuação e emojis, e a ordenação **utf8mb4_unicode_ci**, compatível com múltiplos idiomas. Em seguida, define que o banco de dados a ser usado será esse. Após isso, cria a tabela **users** com quatro colunas: **id** (chave primária com incremento automático), **name** (nome do usuário), **email** (com restrição de unicidade para evitar duplicações) e **created_at** (data de criação automática do registro). Por fim, o script insere dois registros fictícios na tabela, úteis para testes iniciais da API.

Em seguida, realize as seguintes alterações no arquivo **.env**:

```
DB_HOST=localhost
DB_USER=seu_usuario
DB_PASSWORD=sua_senha
DB_DATABASE=projeto_api_db
PORT=3000
```

O arquivo **.env** é utilizado para armazenar variáveis de ambiente do projeto — informações externas ao código, que podem mudar dependendo do ambiente (desenvolvimento, teste, produção). Essas variáveis são carregadas automaticamente no projeto usando a biblioteca **dotenv**.

Em seguida, realize as seguintes alterações no arquivo **src/config/database.js**:

```
const mysql = require("mysql2/promise");
require("dotenv").config();

// Cria a conexão com o banco
const pool = mysql.createPool({
  host: process.env.DB_HOST,           // Endereço do servidor de banco
  user: process.env.DB_USER,           // Usuário para conectar no banco
  password: process.env.DB_PASSWORD,   // Senha do usuário
  database: process.env.DB_DATABASE,   // Nome do banco que será usado
  waitForConnections: true,            // Faz a conexão aguardar caso todas as
  // conexões estejam ocupadas
  connectionLimit: 10,                 // Limita o número máximo de conexões simultâneas
  queueLimit: 0 // Número máximo de requisições enfileiradas (0 = sem limite)
});
module.exports = pool;
```

Observe neste código que criamos um pool de conexões com o banco de dados. Um pool mantém várias conexões abertas e reaproveita elas entre as requisições, o que melhora o desempenho. Limitar o número de conexões (**connectionLimit: 10**) protege o servidor de sobrecarga.

2.3 Configuração do aplicativo Express

O **app.js** é onde configuramos o Express, as rotas e os middlewares — ou seja, a lógica da aplicação. Logo, altere o arquivo **src/app.js**:

```
const express = require('express');
// Importa o framework Express, utilizado para criar o servidor HTTP e
// gerenciar rotas

const cors = require('cors');
// Importa o middleware que permite o compartilhamento de recursos entre
// diferentes origens (Cross-Origin Resource Sharing)

const helmet = require('helmet');
// Importa o middleware de segurança que adiciona cabeçalhos HTTP para proteger
// contra ataques comuns

const userRoutes = require('./routes/userRoutes');
// Importa as rotas relacionadas aos usuários

const errorMiddleware = require('./middlewares/errorMiddleware');
// Importa o middleware para tratamento centralizado de erros

const app = express();
// Cria uma instância do aplicativo Express

// Middlewares globais
app.use(cors());
// Habilita o CORS em todas as rotas da aplicação

app.use(helmet());
// Adiciona proteção automática contra vulnerabilidades HTTP
```

```
app.use(express.json());  
// Permite que o servidor interprete requisições com corpo em formato JSON  
  
// Rotas da aplicação  
app.use('/users', userRoutes);  
// Define que todas as requisições iniciadas com /users serão encaminhadas para  
o arquivo userRoutes  
  
// Middleware de tratamento de erros (deve ser adicionado depois das rotas)  
app.use(errorMiddleware);  
// Middleware que captura e trata erros, enviando respostas ao cliente  
  
module.exports = app;  
// Exporta a aplicação configurada para ser utilizada pelo servidor (server.js)
```

Este código configura a aplicação **Express**, adicionando middlewares importantes: **cors**, que habilita requisições de outros domínios (útil para APIs públicas), **helmet**, que adiciona headers HTTP de segurança contra vulnerabilidades conhecidas da web, e **express.json()**, que permite o recebimento de dados em formato JSON. Ele organiza as rotas da aplicação, definindo que todas as requisições iniciadas com **/users** serão tratadas pelo módulo **userRoutes**. Após configurar as rotas, é adicionado um middleware centralizado para tratamento de erros, garantindo que qualquer problema seja capturado e retornado de maneira padronizada. Por fim, o **app** é exportado para ser utilizado no **server.js**, onde o servidor será iniciado.

Importante: o **errorMiddleware** deve ser o último **app.use()** da cadeia, depois de todas as rotas, para que ele possa capturar os erros que ocorrerem nelas.

2.3 Configuração do servidor

O **server.js** é quem realmente inicializa o servidor e escuta a porta para começar a aceitar requisições. Logo, realize as seguintes alterações no arquivo **server.js**:

```
require('dotenv').config(); // Carrega variáveis de ambiente  
  
const app = require('./src/app'); // Importa o app já configurado (rotas,  
middlewares e tratamento de erros)
```

```
const PORT = process.env.PORT || 3000;

// Inicia o servidor Express para escutar na porta definida
app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

Este trecho de código é responsável por **inicializar o servidor** da aplicação. Primeiro, ele carrega as variáveis de ambiente do arquivo **.env**, permitindo configurar parâmetros como a porta de execução sem alterar o código. Em seguida, importa o aplicativo Express já configurado no **app.js**, define a porta que será usada (pegando do **.env** ou assumindo **3000** por padrão) e inicia o servidor com **listen()**, exibindo no console uma mensagem indicando que a aplicação está rodando corretamente.

2.4 Model

A camada Model é responsável por interagir diretamente com o banco de dados, realizando operações de leitura, gravação, atualização e exclusão, sem aplicar regras de negócio. Neste contexto, realize as seguintes alterações no arquivo **src/models/userModel.js**:

```
const db = require('../config/database');
// Importa a conexão pool com o banco de dados

class UserModel {
  // Busca todos os usuários
  static async findAll() {
    const [rows] = await db.query('SELECT * FROM users');
    return rows;
  }

  // Busca um usuário pelo email
  static async findByEmail(email) {
    const [rows] = await db.query('SELECT * FROM users WHERE email = ?',
    [email]);
    return rows[0];
  }
}
```

```
// Cria um novo usuário
static async create(user) {
  const { name, email } = user;
  const [result] = await db.query('INSERT INTO users (name, email) VALUES (?, ?)', [name, email]);
  return result.insertId; // Retorna o ID do usuário criado
}

// Atualiza um usuário existente
static async update(id, user) {
  const { name, email } = user;
  const [result] = await db.query('UPDATE users SET name = ?, email = ? WHERE id = ?', [name, email, id]);
  return result.affectedRows; // Retorna o número de linhas afetadas
}

// Deleta um usuário pelo ID
static async delete(id) {
  const [result] = await db.query('DELETE FROM users WHERE id = ?', [id]);
  return result.affectedRows; // Retorna o número de linhas afetadas
}

module.exports = UserModel;
// Exporta a classe UserModel para ser usada nos services
```

Este trecho define o **UserModel**, uma classe que centraliza as operações de acesso ao banco de dados relacionadas à tabela **users**. Utilizando a conexão configurada em **database.js**, ele implementa métodos assíncronos para listar todos os usuários, buscar por e-mail, criar novos registros, atualizar informações e excluir usuários. Cada método executa uma consulta SQL correspondente e retorna os dados ou resultados da operação, promovendo a separação clara entre a camada de dados (**Model**) e as outras camadas da aplicação.

2.5 Service

A camada Service é responsável por aplicar as regras de negócio da aplicação. Neste contexto, realize as seguintes alterações no arquivo **src/services/userService.js**:

```
const UserModel = require("../models/userModel");
// Importa o Model responsável pelo acesso ao banco de dados (tabela users)

const validateEmail = require("../utils/validateEmail");
// Importa a função utilitária que valida o formato de e-mail

class UserService {
  // Busca todos os usuários cadastrados
  static async getAllUsers() {
    return await UserModel.findAll();
  }

  // Cria um novo usuário após validações
  static async createUser(user) {
    if (!validateEmail(user.email)) {
      throw new Error("Formato de email inválido."); // Valida o formato do
e-mail
    }

    const existingUser = await UserModel.findByEmail(user.email);
    if (existingUser) {
      throw new Error("Email já cadastrado."); // Impede cadastro de e-mails
duplicados
    }

    return await UserModel.create(user); // Cria o novo usuário
  }

  // Atualiza informações de um usuário existente
  static async updateUser(id, user) {
    const updatedRows = await UserModel.update(id, user);
    if (updatedRows === 0) {
```

```
        throw new Error("Usuário não encontrado."); // Caso nenhum usuário tenha
        sido atualizado
    }
    return updatedRows;
}

// Deleta um usuário pelo ID
static async deleteUser(id) {
    const deletedRows = await UserModel.delete(id);
    if (deletedRows === 0) {
        throw new Error("Usuário não encontrado."); // Caso nenhum usuário tenha
        sido deletado
    }
    return deletedRows;
}
}

module.exports = UserService;
// Exporta a classe para ser utilizada pelos controllers
```

Esta classe **UserService** implementa as regras de negócio para a gestão de usuários. Ela valida o formato do e-mail, impede cadastros duplicados, garante que operações de atualização e exclusão só ocorram se o usuário existir, e organiza o fluxo das chamadas ao banco de dados usando o **UserModel**. Cada método do serviço concentra a lógica antes de qualquer acesso aos dados, separando claramente o que é decisão de negócio do que é simples manipulação de dados.

2.6 Controller

A camada Controller é responsável por receber a requisição HTTP, extrair os dados necessários, chamar o Service adequado e devolver uma resposta ao cliente.

Logo, realize as seguintes alterações no arquivo **src/controllers/userController.js**:

```
const UserService = require('../services/userService');
```

```
// Importa o serviço que contém a lógica de negócio para manipular usuários

class UserController {
  // Método para listar todos os usuários
  static async getAll(req, res) {
    try {
      const users = await UserService.getAllUsers(); // Chama o service para
      buscar usuários
      res.json(users); // Retorna a lista em formato JSON
    } catch (error) {
      res.status(500).json({ error: error.message }); // Em caso de erro,
      retorna status 500 (erro interno)
    }
  }

  // Método para criar um novo usuário
  static async create(req, res) {
    try {
      const id = await UserService.createUser(req.body); // Chama o service
      para criar usuário
      res.status(201).json({ message: 'Usuário criado com sucesso.', id }); //
      Retorna status 201 (criado) e o ID
    } catch (error) {
      res.status(400).json({ error: error.message }); // Em caso de erro de
      validação, retorna status 400
    }
  }

  // Método para atualizar um usuário existente
  static async update(req, res) {
    try {
      const id = req.params.id; // Pega o ID da URL
      await UserService.updateUser(id, req.body); // Chama o service para
      atualizar
      res.json({ message: 'Usuário atualizado com sucesso.' });
    } catch (error) {
```

```
        res.status(400).json({ error: error.message }); // Retorna erro se não
        encontrar ou problema nos dados
    }
}

// Método para deletar um usuário
static async delete(req, res) {
    try {
        const id = req.params.id; // Pega o ID da URL
        await UserService.deleteUser(id); // Chama o service para deletar
        res.json({ message: 'Usuário deletado com sucesso.' });
    } catch (error) {
        res.status(400).json({ error: error.message }); // Retorna erro se
        usuário não encontrado
    }
}
}

module.exports = UserController;
// Exporta o Controller para ser usado nas rotas
```

A classe **UserController** é responsável por gerenciar a comunicação entre as requisições HTTP e a camada de serviços da aplicação. Ela interpreta as requisições recebidas (dados no corpo ou parâmetros na URL), chama os métodos correspondentes do **UserService** para executar a lógica de negócio e envia as respostas HTTP apropriadas de volta ao cliente. Em caso de erro, ela captura a exceção e retorna uma mensagem clara, com o status correto, mantendo a padronização da comunicação da API.

2.7 Rotas

Agora, vamos configurar as rotas de nossa aplicação. Realize as seguintes alterações no arquivo **src/routes/userRoutes.js**:

```
const express = require('express');
// Importa o framework Express
```

```
const UserController = require('../controllers/userController');
// Importa o controller responsável por gerenciar as ações de usuário

const router = express.Router();
// Cria uma nova instância de roteador do Express

// Define a rota para listar todos os usuários
router.get('/', UserController.getAll);
// Define a rota para criar um novo usuário
router.post('/', UserController.create);
// Define a rota para atualizar um usuário existente pelo ID
router.put('/:id', UserController.update);
// Define a rota para deletar um usuário pelo ID
router.delete('/:id', UserController.delete);

module.exports = router;
// Exporta o roteador configurado para ser usado no app principal
```

Este código define as rotas relacionadas à entidade "**usuário**" utilizando o roteador do Express. Cada rota é associada a um método específico do **UserController**, que gerencia a ação correspondente, como listar, criar, atualizar ou excluir usuários. As rotas são configuradas de forma RESTful, usando os métodos HTTP adequados (GET, POST, PUT, DELETE). O roteador é exportado para ser incorporado no **app.js**, organizando e modularizando o acesso às funcionalidades da API.

2.8 Middleware de Tratamento de Erros

Vamos adicionar um middleware para unificar o tratamento de erros da aplicação. Realize as seguintes alterações no arquivo **src/middlewares/errorMiddleware.js**:

```
// Middleware para captura de erros em toda a aplicação
function errorMiddleware(err, req, res, next) {
  console.error('[ERRO]:', err.stack);
  // Exibe o erro completo no console para depuração

  const statusCode = err.statusCode || 500;
```

```
// Define o código de status da resposta: usa o informado no erro ou 500
(erro interno) como padrão

res.status(statusCode).json({
  error: err.message || 'Erro interno no servidor',
  status: statusCode
  // Retorna uma resposta JSON padronizada com a mensagem de erro e o status
});
}
module.exports = errorMiddleware;
// Exporta o middleware para ser utilizado no app.js
```

O **errorMiddleware** é um middleware global responsável por capturar e tratar todos os erros que ocorrem durante o processamento das requisições. Quando um erro é lançado em qualquer parte da aplicação, ele é interceptado por esse middleware, que registra a falha no console para depuração e envia uma resposta JSON padronizada ao cliente, contendo o código de status apropriado e a mensagem de erro. Isso garante que a API tenha um tratamento de erros centralizado e consistente, melhorando a segurança e a experiência de uso.

2.9 Utils

A pasta **utils/** serve para armazenar funções utilitárias — ou seja, códigos auxiliares que não pertencem diretamente a uma camada (controller, service, model), mas que são úteis em várias partes da aplicação. Realize as seguintes alterações no arquivo **src/utils/validateEmail.js**:

```
// Função para validar se o email tem um formato válido
function validateEmail(email) {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  // Expressão regular que verifica se o email tem um formato padrão:
  texto@texto.texto
  return regex.test(email);
  // Retorna true se o email for válido conforme o padrão, ou false se não for
}
module.exports = validateEmail;
// Exporta a função para ser utilizada em outros módulos, como nos services
```

A função **validateEmail** verifica se um e-mail enviado segue um formato válido, utilizando uma expressão regular. Ela confere se o e-mail possui texto antes e depois do símbolo @, e se termina com um domínio válido, como .com ou .org. A função retorna **true** para e-mails corretamente formatados e **false** para e-mails inválidos, sendo essencial para validar entradas de usuários e evitar erros ou dados inconsistentes no sistema.

2.10 Database

A pasta **database/** tem como função organizar tudo que é relacionado à estrutura e configuração do banco de dados, separado do restante da aplicação. Mesmo já tendo executado estes comandos SQL em nosso exemplo, vamos organizá-los adequadamente.

2.10.1. Script para criar o banco de dados

Realize as seguintes alterações no arquivo **database/create_database.sql**:

```
-- Cria o banco de dados se não existir
CREATE DATABASE IF NOT EXISTS projeto_api_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

-- Usa o banco de dados
USE projeto_api_db;
```

2.10.2. Script para criar a tabela users

Realize as seguintes alterações no arquivo **database/create_tables.sql**:

```
-- Usa o banco correto
USE projeto_api_db;

-- Cria a tabela de usuários
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.10.3. Script para inserir dados de exemplo

Realize as seguintes alterações no arquivo **database/seed_data.sql**:

```
-- Usa o banco
USE projeto_api_db;

-- Inserção de dados fictícios
INSERT INTO users (name, email) VALUES
('João da Silva', 'joao@example.com'),
('Maria Oliveira', 'maria@example.com');
```

2.11 Testando as Rotas do Servidor HTTP

Primeiramente, assegure que o servidor que está desenvolvendo esteja rodando. Para executar o servidor usando o Nodemon, digite o seguinte comando:

```
npm run dev
```

Neste curso, utilizamos a extensão [REST Client do VS Code](#) para testar os endpoints da aplicação. Neste contexto, realize as seguintes alterações no arquivo **tests/users_api_test.rest**:

```
@baseUrl = http://localhost:3000
@id_do_usuario = 3

### Listar todos os usuários
GET {{baseUrl}}/users
Accept: application/json

### Criar um novo usuário
POST {{baseUrl}}/users
Content-Type: application/json

{
  "name": "Carlos Almeida",
  "email": "carlos@example.com"
}
```

Criar usuário com email inválido (erro esperado)

POST {{baseURL}}/users

Content-Type: application/json

```
{
  "name": "Teste Email Invalido",
  "email": "email_invalido"
}
```

Criar usuário com email repetido (erro esperado)

POST {{baseURL}}/users

Content-Type: application/json

```
{
  "name": "Carlos Almeida",
  "email": "carlos@example.com"
}
```

Atualizar um usuário existente (substitua @id_do_usuario)

PUT {{baseURL}}/users/{{id_do_usuario}}

Content-Type: application/json

```
{
  "name": "Carlos de Almeida Atualizado",
  "email": "carlosatualizado@example.com"
}
```

Atualizar usuário inexistente (erro esperado)

PUT {{baseURL}}/users/9999

Content-Type: application/json

```
{
  "name": "Usuario Inexistente",
  "email": "inexistente@example.com"
}
```

```
### Deletar um usuário existente (substitua @id_do_usuario)
```

```
DELETE {{baseURL}}/users/{{id_do_usuario}}
```

```
Accept: application/json
```

```
### Deletar um usuário inexistente (erro esperado)
```

```
DELETE {{baseURL}}/users/9999
```

```
Accept: application/json
```

Execute estes testes e verifique que nossa aplicação está funcionando.

Referências

O material desenvolvido para este capítulo baseou-se nas seguintes obras:

- ALVES, W. P. Projetos de sistemas Web: conceitos, estruturas, criação de banco de dados e ferramentas de desenvolvimento. São Paulo: Érica, 2019.
- GAMMA, Erich et al. Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos. Rio de Janeiro: Bookman, 2000.
- MICROSOFT. GitHub REST API Best Practices. Disponível em: <https://docs.microsoft.com/en-us/azure/architecture/best-practices/api-design>. Acesso em: 29 abr. 2025.
- SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.